Scorecard Heuristic

Branching heuristic using repeated randomized subinstance solving

By: Vivek Mehta

Preface :)

In the past year I've sporadically worked on SAT-related stuff, and I spent a decent bit of time building my own solver which helped a lot to learn how they work (and how to make them faster).

Later, I pivoted to looking at optimizing solvers and why heuristics (specifically branching) are so effective.

I remembered how ***shotgun sequencing*** was the critical breakthrough that made sequencing the human genome possible. And so the parallels between variables and nucleotides sparked the following idea!

I'm also not sure if this is a new approach, but from what I've seen there isn't a heuristic technique that does the same thing that I do (to my knowledge). Thank you for taking the time to read this!

At a high level (the tl;dr)

Simultaneous polarity and variable selection through repeated **randomized subinstance solving

1. Separate clauses into base and additional clauses
2. **F = B + A** (Almost always **|A|** is much bigger than **|B|**)
3. Initialize blank scorecard
4. Repeat **N** times:
    a. Create a **F' = B + A'**, where **A'** is some random **P%** of **A**
    b. Solve **F'** (if we assume **F** is **SAT**, **F' ⊆ F** is **SAT**)
    c. Use resulting solution **alpha** to update scorecard
5. Pass scorecard to modified solver
6. Find best variable
    a. Variable selection from **max(fabs(score))** out of all scores
    b. Polarity selection from **sign(score)**
7. If invalid or fabs(score[x]) not above threshold, default to VSIDS
    a. Otherwise, branch on that variable with polarity we found
8. Solve F



***Notice that the base clauses remain consistent*

Which decision problem?

- Decision problems that have “placing/marking” fit well
- Originally worked with **Costas Arrays**
  - **Lots of solutions (lots of symmetry as well)**
- However, a smaller # of valid solutions vastly improves scorecard effectiveness $\Longrightarrow$ Pivoted to the 1D version, the **Golomb Ruler**!

Leveraged PySAT for CNF and scorecard generation, locally cloned & modified **MiniSAT** to implement the modified branching heuristic.

Relevant Code Snippets

```python
def generateScorecard(length, baseClauses, additionalClauses):
    scorecard = [0.0] * length
    start = time.perf_counter()
    for _ in range(numIterations):
        randF = CNF()
        randF.extend(baseClauses)

        # Select random subset of constraint clauses
        sampled = random.sample(
                    additionalClauses,
                    int(len(additionalClauses)*pctRandom/100)
                )
        randF.extend(sampled)

        r, _ = golombSAT(randF, False)
        alpha = rulerToBoard(r)
        scorecard = updateBoard(scorecard, alpha)
    end = time.perf_counter()
    generationTime = end - start
    return scorecard, generationTime
```

If **FALSE**, subtract X, if **TRUE** add Y
Scorecard is globally updated
X and Y are constants

```
Lit Solver::pickBranchLit() {
    Lit l = pickBranchLit_ScorecardHeuristic();

    if (l != lit_Undef) return l;
    else return pickBranchLit_VSIDS();
}

Lit Solver::pickBranchLit_ScorecardHeuristic() {
    . . .
    double bestScore = THRESHOLD;
    iterate over all x
            if x invalid continue;
           double score = fabs(scorecard[v]);

            if (score > bestScore) {
                bestScore = score; bestPick = x;
            }
        }
    }

    if (bestPick != var_Undef) {
        Lit pickedLit = mkLit(bestPick, scorecard[ . . . ] < 0);
        return pickedLit;
    }

    return lit_Undef;
}

Lit Solver::pickBranchLit_VSIDS() {
    //original MiniSAT code
}
```

Results

(time in seconds)

comparison of total times

scorecard generation

SAT solving w/ scorecard

```
(venv) vivekmehta@Mac satsolver % python3 sat.py
Ruler:   [0, 4, 5, 17, 28, 31, 50, 60, 66, 68, 75]

base clauses: 57771, additional clauses: 268125

Generation done!

Ruler:   [0, 6, 8, 26, 31, 38, 47, 60, 71, 74, 75]

NORMAL SOLVE TIME:                  7.193725958000869
HEURISTIC SOLVE TIME:               3.3133613319369033
└-> SPLIT:                          1.8338789159897715 + 1.4794824159471318
```

RULER OF SIZE 11, upper bounded length of 75.

5 Iterations, 93% random, trueWeight = 5.5, falseWeight = 1.5

Results (cont.)

comparison of total times

scorecard generation

SAT solving w/ scorecard

```
(venv) vivekmenta@Mac satsolver % python3 sat.py
Ruler:   [0, 4, 18, 26, 35, 37, 38, 62, 67, 77, 83, 90]

base clauses: 91103, additional clauses: 463320

[##################################################] 10/10
Generation done!

time:  414.0716102090664
Ruler:   [0, 4, 18, 26, 35, 37, 38, 62, 67, 77, 83, 90]

NORMAL SOLVE TIME:                515.6960611250252
HEURISTIC SOLVE TIME:             441.8915781250689
└-> SPLIT:                        27.819967916002497 + 414.0716102090664
```

RULER OF SIZE 12, upper bounded length of 90.

10 Iterations, 97% random, trueWeight = 7.0, falseWeight = 2.0

Quips and queries

- The **heuristic is consistently better than normal** — yay!
- Worthwhile to test on problems that are not the Golomb ruler
- **pctRandom matters much more than numIterations**
  - Even a few iterations of a high % solve is very effective
- The scorecard is meant to emulate a “heatmap” of solutions
- Can definitely be optimized more to be faster

- Can the scorecard be adapted to higher dimension spaces?
- Is there a way to find the “best” constants rigorously?
  - Threshold could be a f(scorecard) (e.g. using the avg)
  - What should trueWeightage/falseWeightage be?

Limitations

- The heuristic is static (generated once, used many times)
  - Defaulting to VSIDS (a dynamic heuristic) to help combat this
- Hard to decide which clauses are base or not
  - What if subinstance and full solutions don’t have good correlation?
- Heuristic could be biased towards what is easy to SAT
  - But could miss very niche, hard, important constraints
- Not good for UNSAT instances
- Encoding could always be better
  - Could provide faster generation time and thus, faster solve time

Future work:

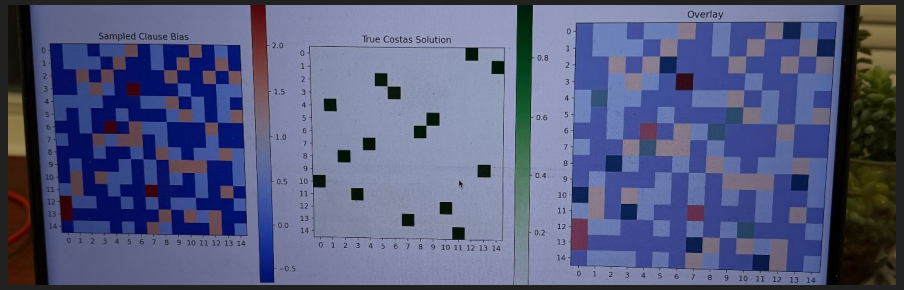
- I think there's something interesting here, but would love to hear your thoughts
  - My main concern is unintentionally reinventing the same heuristic
  - Although, I suspect there might be a lot of relevant/similar work
  - Still an early idea, and can definitely be shaped into something cool!
- **Not sure how to go about to formalizing this?**
  - Similar to Monte-Carlo like algorithms, could look there for inspiration
- **Could case on the "depth" of the search tree**
  - Similar to epsilon-greedy algorithm
  - **Explore**: pick variables early on with heuristic
  - **Exploit**: solve a reduced CNF dynamically w/ VSIDS
- **How much numIter & pctRand can be decreased w/o compromising time?**
- Could integrate VSIDS directly as a hybrid
  - Something like f(x) = a * VSIDS(x) + b * |scorecard(x)|

- Very very interested in any ideas, concerns, advice, etc that you may have! :)

Codebase Clarifications

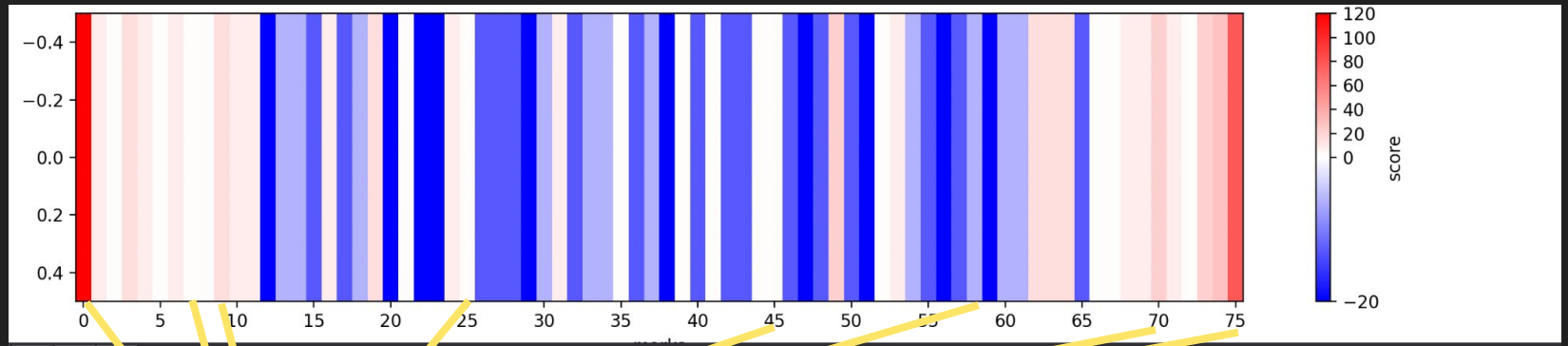
- For the proof of concept, the whole pipeline is not fully automated, instead it looks something like this:
  - Run the python script and generate the scorecard and break early
  - Hardcode the scorecard in the modified MiniSAT solver
  - Run the full script (which runs the modified MiniSAT executable w/ subprocess)
  - Solve and print times
- The codebase is not streamlined due to testing, but I’m happy to provide

Miscellaneous progress photos:



Less correlation w/ Costas, green is just one of MANY solutions

Generated scorecard!



More red = % mark higher!

Actual solution!

```
Full CNF time: 38.023326 6597414 seconds

Ruler: [0, 7, 9, 15, 25, 44, 47, 58, 70, 71, 75]
Valid? True
base clauses: 57771, additional clauses: 268125

[##################################################] 20/20
Heuristic Generation time: 16.316337541968096 seconds
```

thanks for reading !!